
Logical Clocks

Event Ordering

- When there is no common memory or clock, it is sometimes impossible to say which of two events occurred first.
- The *happened-before* relation is a **partial ordering** of events in distributed systems such that
 - 1 If A and B are events in the same process, and A was executed before B , then $A \Rightarrow B$.
 - 2 If A is the event of sending a message by one process and B is the event of receiving that by another process, then $A \Rightarrow B$.
 - 3 If $A \Rightarrow B$ and $B \Rightarrow C$, then $A \Rightarrow C$.
- If two events A and B are not related by the $\Rightarrow$ relation, then they are executed concurrently (no causal relationship)

Ordering of Events

Lamport's *Happened Before* relationship:

For two events a and b , $a \rightarrow b$ if

- *a and b are events in the same process and a occurred before b , or*
- *a is a send event of a message m and b is the corresponding receive event at the destination process, or*
- *$a \rightarrow c$ and $c \rightarrow b$ for some event c*

Causally related events and concurrent events

- **Causally related events:** Event a causally affects event b if $a \rightarrow b$.
- **Concurrent events:** Two different events a and b are concurrent (denoted $a \parallel b$) if a does not causally affect b and b does not causally affect a .
- For any two events a and b , either $a \rightarrow b$ or $b \rightarrow a$ or $a \parallel b$.

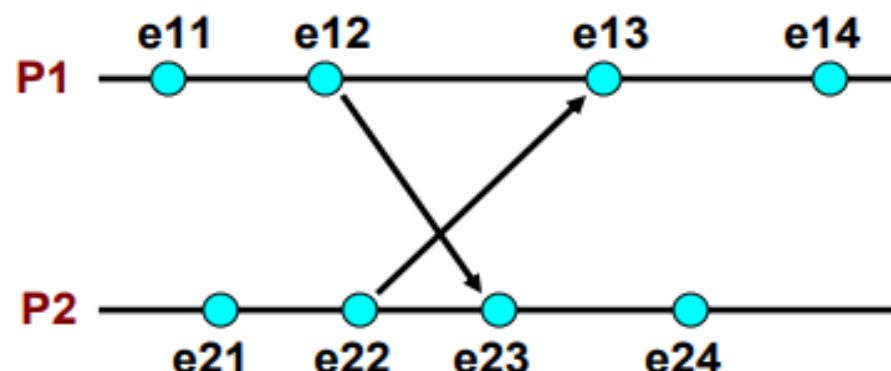
Causally Related versus Concurrent

Causally related events:

Event a causally affects event b if $a \rightarrow b$

Concurrent events:

- Two distinct events a and b are said to be concurrent (denoted by $a||b$) if $a \not\rightarrow b$ and $b \not\rightarrow a$



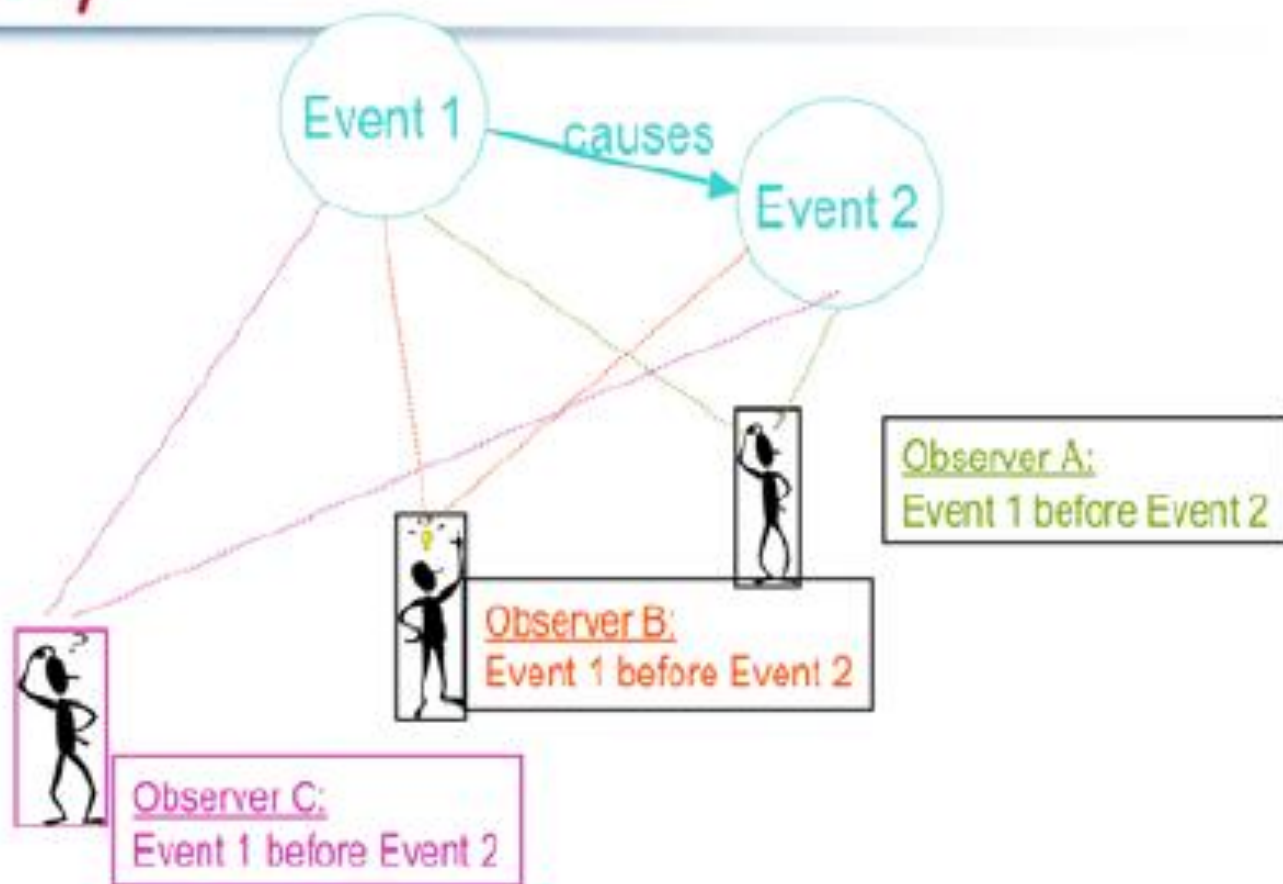
e11 and e21 are concurrent

e14 and e23 are concurrent

e22 causally affects e14

A space-time diagram

Causality



Requirement: We have to establish causality, i.e., each observer must see event 1 before event 2

Event Timelines (Example of previous Slide)



Note: In the timeline view, event 2 must be caused by some passage of information from event 1 if it is caused by event 1

Lamport's logical clock

- There is a clock C_i at each process P_i .
- The clock is a function that assigns a number $C_i(a)$ to any event a called the timestamp of event a in process P_i .
- It has no connection with physical time.
- It assumes monotonically increasing values.

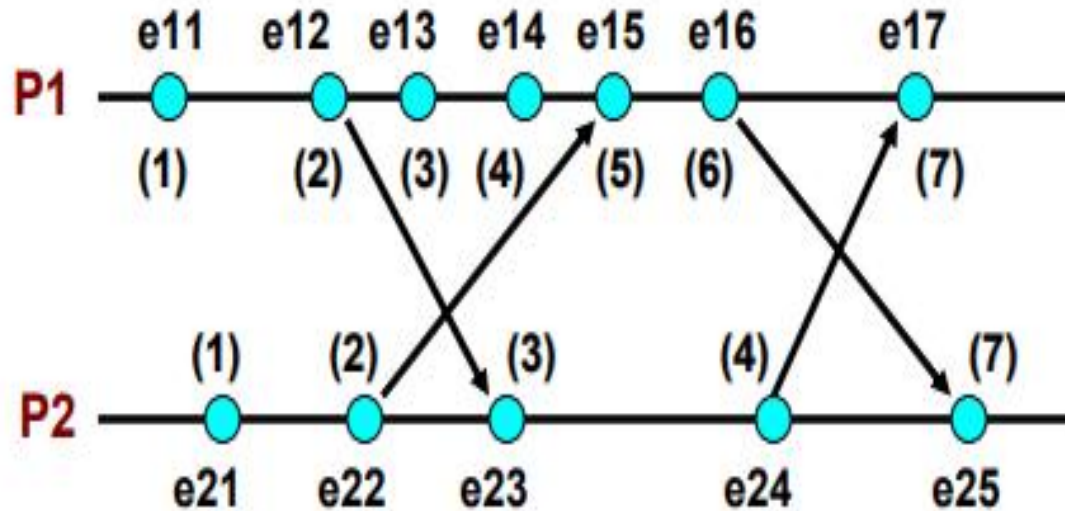
Lamport's Logical Clock

Each process i keeps a clock C_i

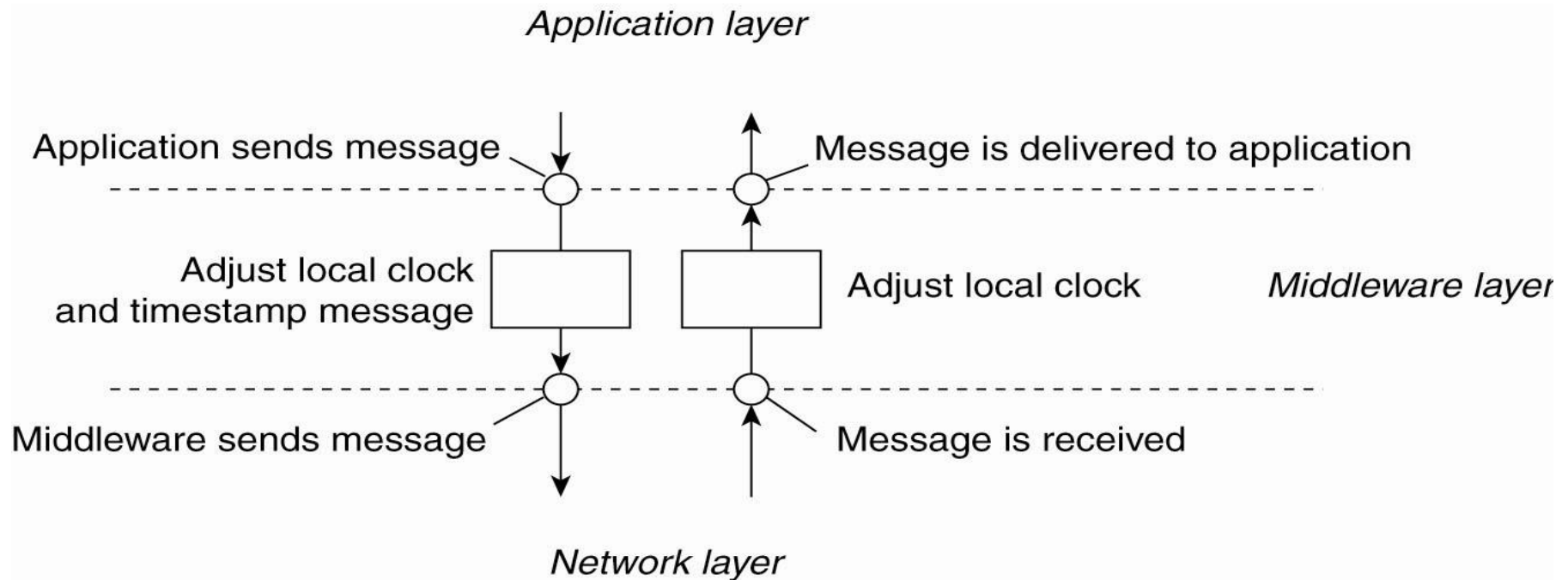
- Each event a in i is time-stamped $C_i(a)$, the value of C_i when a occurred
- C_i is incremented by 1 for each event in i
- In addition, if a is a send of message m from process i to j , then on receive of m ,

$$C_j = \max (C_j, C_i(a)+1)$$

How Lamport's clocks advance



The positioning of Lamport's logical clocks in distributed systems.



Conditions satisfied by logical clocks

- For any events a and b
 - if $a \rightarrow b$, then we want $C(a) < C(b)$.
- The above is possible if logical clocks obey the following 2 conditions

[C1] For any 2 events a and b in a process P_i , if a occurs before b , then $C_i(a) < C_i(b)$.

[C2] If a is the event of sending a message m in P_i and b is the event of receiving the same message m at process P_j , then $C_i(a) < C_j(b)$.

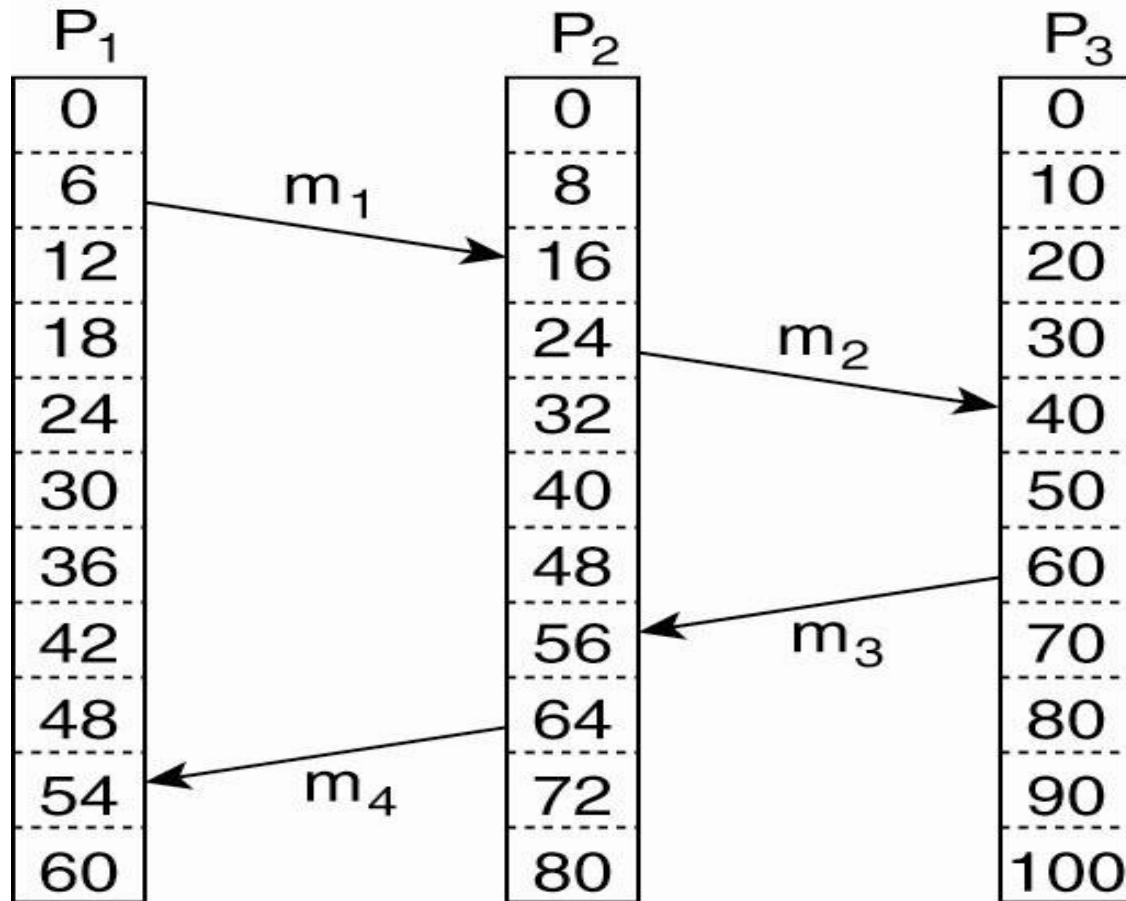
Implementation rules of logical clocks

- The following implementation rules guarantee the satisfaction of conditions C_1 and C_2 :
 - **[IR1]** Clock C_i is incremented between any two successive events (local event or message send/receive) in process P_i :
 - $C_i := C_i + d$ ($d > 0$)If a and b are 2 successive events in P_i and $a \rightarrow b$, then $C_i(b) = C_i(a) + d$
 - **[IR2]** If event a is the sending of message m by process P_i , then message m is assigned the timestamp $t_m = C_i(a)$. On receiving the same message m by process P_j , C_j is set as:
 - $C_j := \max(C_j, t_m + d)$ ($d > 0$)

d is usually set to 1.
- N.B. First apply [IR1]. Then apply [IR2].

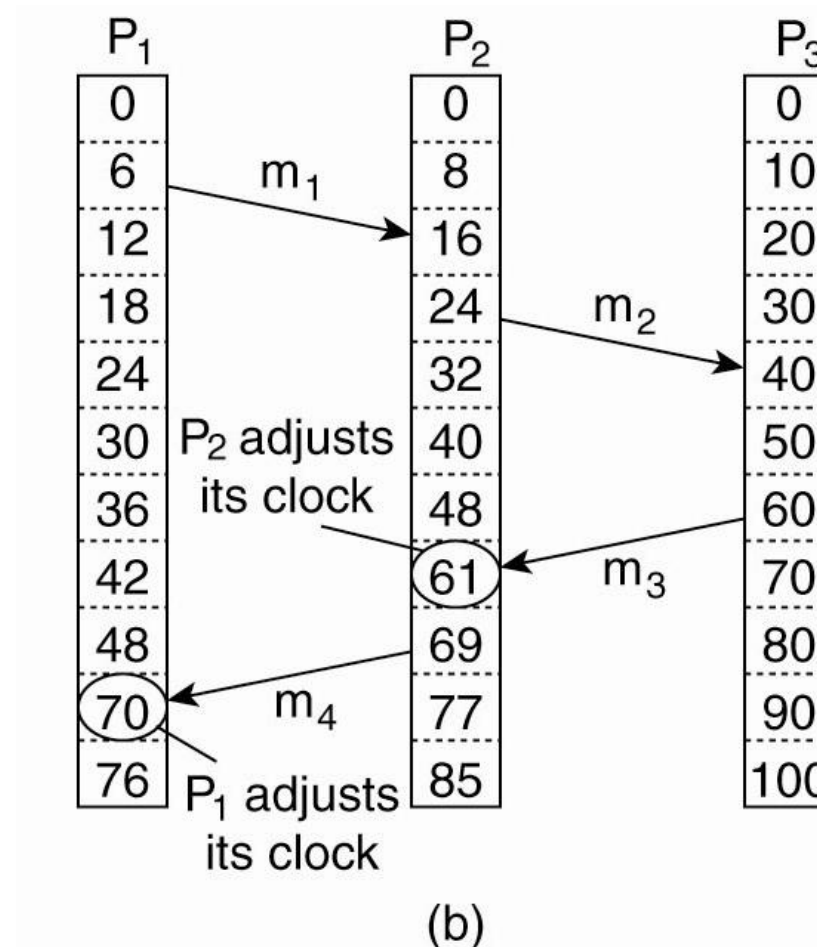
Lamport's Logical Clocks

- Three processes, each with its own clock. The clocks run at different rates.



(a)

Lamport's Logical Clocks



■ Lamport's algorithm corrects the clocks.

Basic Properties

■ Consistency Property

Scalar clocks satisfy the monotonicity and hence the consistency property for two events e_i and e_j

$$e_i \rightarrow e_j \Rightarrow C(e_i) < C(e_j)$$

■ Total Ordering

Scalar clocks can be used to totally order events in a distributed system. The main problem in totally ordering events is that two or more events at different processes may have identical timestamp.

Total Ordering

- Total order is a binary relation that can be used to compare any 2 elements of a set with each other. As a direct consequence of this property, using this relation we can derive only a single order for all the elements in a set, which is why it's called total order.
- The set of unique integers $\{7, 9, 3, 2, 6\}$ can be totally ordered (using the relation less than $<$) and the associated total order is
[2, 3, 6, 7, 9].

Partial order

- Partial order is a binary relation that can be used to compare only some of the elements of a set with each other. As a consequence, using this relation we can derive multiple, valid orders for all the elements in the set.

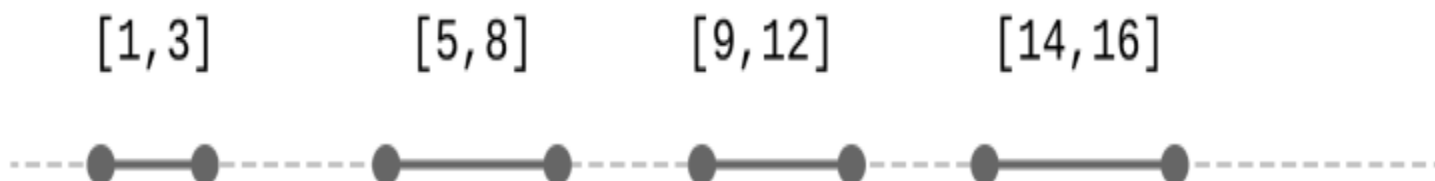
For example, the set of the following sets of integers $\{\{0\}, \{1\}, \{2\}, \{0,1\}, \{1,2\}\}$ can only be partially ordered, using the subset relation $\subseteq$. If we pick the elements $\{0\}$ and $\{0,1\}$, then they can be ordered, since $\{0\} \subseteq \{0,1\}$.

The elements $\{0,1\}$ and $\{1,2\}$ cannot be ordered using this relation, since neither $\{0,1\} \subseteq \{1,2\}$ nor $\{1,2\} \subseteq \{0,1\}$ holds. As a result, both $[\{0\}, \{1\}, \{2\}, \{0,1\}, \{1,2\}]$ and $[\{2\}, \{1\}, \{0\}, \{1,2\}, \{0,1\}]$ would be valid partial orderings.

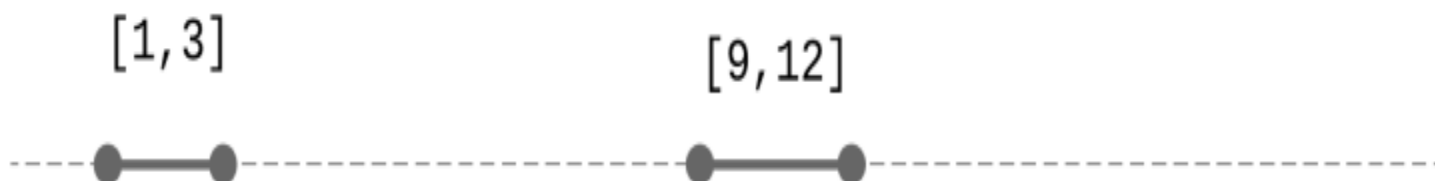
Conclusion:

- In systems that are composed of a single node, it's easy and intuitive to determine a total ordering of events. The main reason is that there is a single actor, where all the events happen, so this actor can impose a total order on these events as they occur.
 - Total orderings also make it much simpler to build protocols and algorithms.
 - However, in a distributed system it's not that straightforward to impose a total order on events, since there are multiple nodes in the system and events might be happening concurrently on different nodes.
-

SINGLE
NODE



NODE A



NODE B



NODE C



Partial order on events

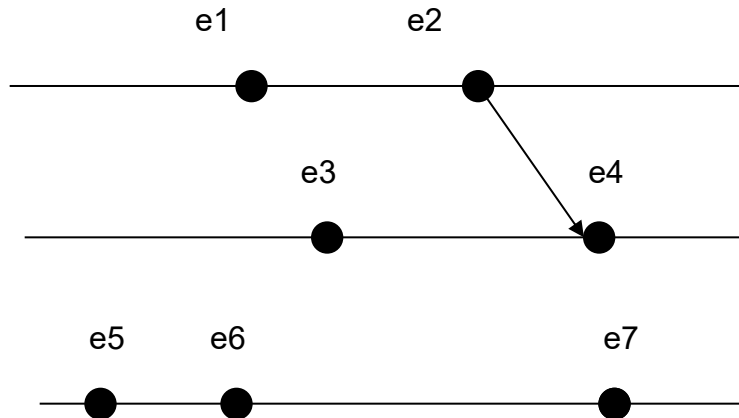
- The system of scalar clocks is not strongly consistent; that is, for two events e_i and e_j , $C(e_i) < C(e_j) \neq \Rightarrow e_i \rightarrow e_j$.

For example, the first event of process P_1 may have smaller scalar timestamp than the 2nd event of process P_3 . However, the former did not happen before the latter.

- The reason that scalar clocks are not strongly consistent is that the logical local clock and logical global clock of a process are squashed into one, resulting in the loss causal dependency information among events at different processes.

Limitation of Lamport's clocks

- If $a \rightarrow b$, $C(a) < C(b)$
- The reverse is not necessarily true



Timestamp of e1 = (1).

Timestamp of e2 = (2)

e1 is causally related to e2

Timestamp of e6 = (2)

$C(e1) < C(e6)$ but e1 is not causally related to e6.

Points to note

- if $a \rightarrow b$, then $C(a) < C(b)$
- $\rightarrow$ is a partial order
- Total ordering possible by arbitrarily ordering concurrent events by process numbers

Limitation of Lamport's Clock

$a \rightarrow b$ implies $C(a) < C(b)$

BUT

$C(a) < C(b)$ doesn't imply $a \rightarrow b$!!

So not a true clock !!

Motivation for Vector Clocks

- Logical clocks induce an order consistent with causality, but
 - the converse of the clock condition does not hold: it may be that $LC(e_1) < LC(e_2)$ even if e_1 and e_2 are concurrent.
 - If A could know anything B knows, then it must be $LC_A > LC_B$
 - But if $LC_A > LC_B$ then this doesn't make it so; i.e., "false positives".
 - Concurrent updates may be ordered unnecessarily.
- We need a clock mechanism that is necessary and sufficient in capturing causality.

Solution: Vector Clocks

Each process P_i has a clock C_i , which is a vector of size n

The clock C_i assigns a vector $C_i(a)$ to any event a at P_i

Update rules:

- $C_i[i]++$ for every event at process i
- If a is send of message m from i to j with vector timestamp t_m , then on receipt of m :
$$C_j[k] = \max(C_j[k], t_m[k]) \text{ for all } k$$

Vector Time

- In the system of n vector clocks, the time domain is represented by a set of n -dimensional non-negative integer vectors.
- Each process p_i maintains a vector $vt_i [1..n]$, where $vt_i [i]$ is the local logical clock of p_i and describes the logical time progress at process p_i .
- $vt_i [j]$ represents process p_i 's latest knowledge of process p_j 's local time.
- If $vt_i [j] = x$, then process p_i knows that local time at process p_j has progressed till x .
- The entire vector vt_i constitutes p_i 's view of the global logical time and is used to timestamp events.

Vector Clocks

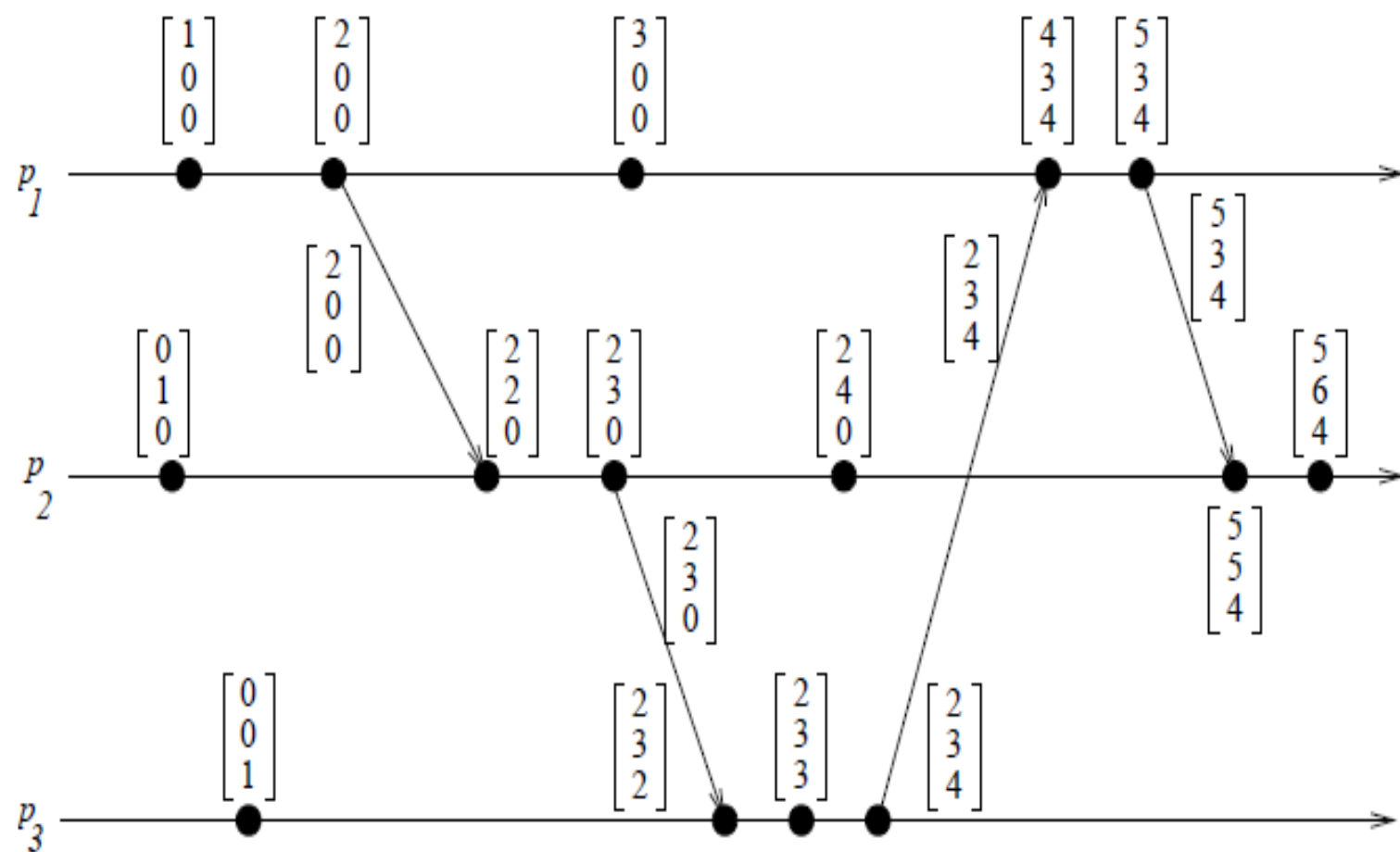
- Let there be n processes in a distributed system
- Each process P_i has a clock C_i which is a vector of length n .
Vector clocks are constructed by letting each process P_i maintain a vector VC_i with the following two properties:
 1. $VC_i[i]$ is the number of events that have occurred so far at P_i . In other words, $VC_i[i]$ is the local logical clock at process P_i .
 2. If $VC_i[j] = k$ then P_i knows that k events have occurred at P_j . It is thus P_i 's knowledge of the local time at P_j .

Vector clocks: Implementation

- The implementation rules follow:
 - [IR1] Clock C_i is incremented between any 2 successive events (local event or message send/receive) in process P_i
 - $C_i[l] := C_i[l] + d$ ($d > 0$)
 - [IR2]
 - If event a is the sending of message m by process P_i , then message m is assigned the timestamp $t_m := C_i(a)$
 - When message m is received by process P_j , then C_j is updated as for all k , $C_j[k] := \max(C_j[k], t_m[k])$
- **Comments:** For each local event and each message send/receive, the logical clock advances. Each message carries as its timestamp the current vector clock value of the sender. The receiver updates its vector clock using the timestamp of the received message.
- **N.B.** First apply [IR1]. Then apply [IR2].

Vector Time

An Example of Vector Clocks



Partial Order between Timestamps

For events a and b with vector timestamps t^a and t^b ,

- Equal: $t^a = t^b$ iff $\forall i, t^a[i] = t^b[i]$
- Not Equal: $t^a \neq t^b$ iff $\exists i, t^a[i] \neq t^b[i]$
- Less or equal: $t^a \leq t^b$ iff $\forall i, t^a[i] \leq t^b[i]$
- Not less or equal: $t^a \not\leq t^b$ iff $\exists i, t^a[i] > t^b[i]$
- Less than: $t^a < t^b$ iff $(t^a \leq t^b \text{ and } t^a \neq t^b)$
- Not less than: $t^a \not< t^b$ iff $\neg(t^a \leq t^b \text{ and } t^a \neq t^b)$
- Concurrent: $t^a \parallel t^b$ iff $(t^a \not< t^b \text{ and } t^b \not< t^a)$

Causal Ordering

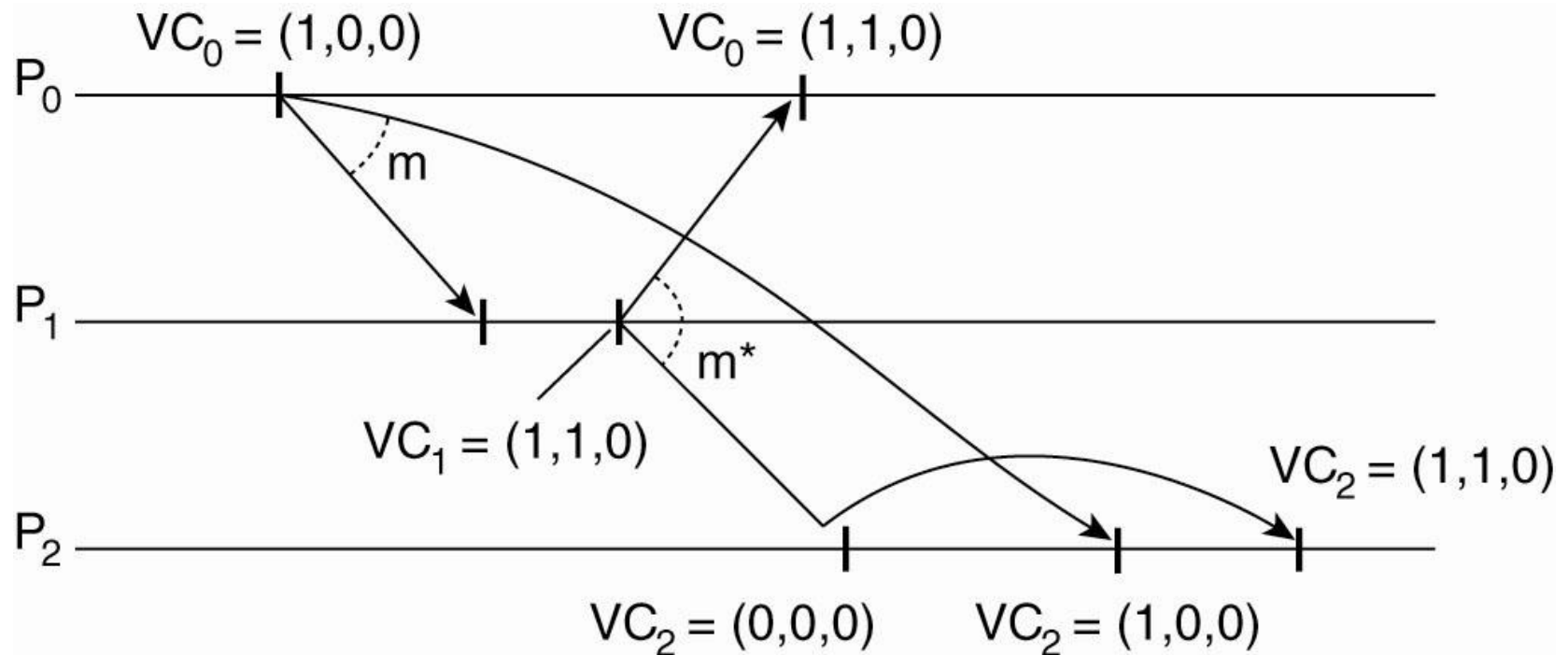
- $a \rightarrow b$ iff $t_a < t_b$
 - Events a and b are causally related iff $t_a < t_b$ or $t_b < t_a$, else they are concurrent
 - Note that this is still not a total order
 - Vector clocks allow us to order events by simply looking at their timestamps.
-

Use of Vector Clocks in Causal Ordering of Messages

- If $\text{send}(m1) \rightarrow \text{send}(m2)$, then every recipient of both message $m1$ and $m2$ must “deliver” $m1$ before $m2$.
 - “deliver” – when the message is actually given to the application for processing

Enforcing Causal Communication

- Enforcing causal communication.



Enforcing Causal communication

- P_j receives a message m from P_i . The delivery of the message to the application layer will then be delayed until the following conditions are met.

$$1. ts(m)[i] = VC_j[i] + 1$$

$$2. ts(m)[k] \leq VC_j[k] \text{ for all } k \neq i$$

- ✓ The first condition states that m is the next message that P_j was expecting from P_i .
- ✓ The second condition states that P_j has seen all the messages that have been seen by P_i . When it sent message m there is no need for process P_j to delay the delivery of its own messages.

Strong Consistency

- The system of vector clocks is strongly consistent; thus, by examining the vector timestamp of two events, we can determine if the events are causally related.
- However, Charron-Bost showed that the dimension of vector clocks cannot be less than n , the total number of processes in the distributed computation, for this property to hold.

Event Counting

- If $d=1$ (in rule $R1$), then the i^{th} component of vector clock at process p_i , $vt_i[i]$, denotes the number of events that have occurred at p_i until that instant.
- So, if an event e has timestamp vh , $vh[j]$ denotes the number of events executed by process p_j that causally precede e . Clearly, $\sum vh[j] - 1$ represents the total number of events that causally precede e in the distributed computation.

Problem of Vector Clock

- Message size increases since each message needs to be tagged with the vector
- Size can be reduced in some cases by only sending values that have changed

Efficient Implementations of Vector Clocks

- If the number of processes in a distributed computation is large, then vector clocks will require piggybacking of huge amount of information in messages.
- The message overhead grows linearly with the number of processors in the system and when there are thousands of processors in the system, the message size becomes huge even if there are only a few events occurring in few processors.
- We discuss an efficient way to maintain vector clocks.
- Charron-Bost showed that if vector clocks have to satisfy the strong consistency property, then in general vector timestamps must be at least of size n , the total number of processes.
- However, optimizations are possible and next, and we discuss a technique to implement vector clocks efficiently.

Singhal-Kshemkalyani's Differential Technique

- *Singhal-Kshemkalyani's differential technique* is based on the observation that between successive message sends to the same process, only a few entries of the vector clock at the sender process are likely to change.
- When a process p_i sends a message to a process p_j , it piggybacks only those entries of its vector clock that differ since the last message sent to p_j .
- If entries $i_1, i_2, \dots, i_{n_1}$ of the vector clock at p_i have changed to $v_1, v_2, \dots, v_{n_1}$, respectively, since the last message sent to p_j , then process p_i piggybacks a compressed timestamp of the form:

$$\{(i_1, v_1), (i_2, v_2), \dots, (i_{n_1}, v_{n_1})\}$$

to the next message to p_j .

Singhal-Kshemkalyani's Differential Technique

When p_j receives this message, it updates its vector clock as follows:

$$vt_i[i_k] = \max(vt_i[i_k], v_k) \text{ for } k = 1, 2, \dots, n_1.$$

- Thus this technique cuts down the message size, communication bandwidth and buffer (to store messages) requirements.
- In the worst of case, every element of the vector clock has been updated at p_i since the last message to process p_j , and the next message from p_i to p_j will need to carry the entire vector timestamp of size n .
- However, on the average the size of the timestamp on a message will be less than n .

Singhal-Kshemkalyani's Differential Technique

- Implementation of this technique requires each process to remember the vector timestamp in the message last sent to every other process.
- Direct implementation of this will result in $O(n^2)$ storage overhead at each process.
- Singhal and Kshemkalyani developed a clever technique that cuts down this storage overhead at each process to $O(n)$. The technique works in the following manner:
- Process p_i maintains the following two additional vectors:
 - ▶ $LS_i[1..n]$ ('Last Sent'):
 $LS_i[j]$ indicates the value of $vt_i[i]$ when process p_i last sent a message to process p_j .
 - ▶ $LU_i[1..n]$ ('Last Update'):
 $LU_i[j]$ indicates the value of $vt_i[i]$ when process p_i last updated the entry $vt_i[j]$.
- Clearly, $LU_i[i] = vt_i[i]$ at all times and $LU_i[j]$ needs to be updated only when the receipt of a message causes p_i to update entry $vt_i[j]$. Also, $LS_i[j]$ needs to be updated only when p_i sends a message to p_j .

Singhal-Kshemkalyani's Differential Technique

- Since the last communication from p_i to p_j , only those elements of vector clock $vt_i[k]$ have changed for which $LS_i[j] < LU_i[k]$ holds.
- Hence, only these elements need to be sent in a message from p_i to p_j . When p_i sends a message to p_j , it sends only a set of tuples

$$\{(x, vt_i[x]) | LS_i[j] < LU_i[x]\}$$

as the vector timestamp to p_j , instead of sending a vector of n entries in a message.

- Thus the entire vector of size n is not sent along with a message. Instead, only the elements in the vector clock that have changed since the last message sent to that process are sent in the format $\{(p_1, latest_value), (p_2, latest_value), \dots\}$, where p_i indicates that the p_i th component of the vector clock has changed.
- This technique requires that the communication channels follow FIFO discipline for message delivery.

Singhal-Kshemkalyani's Differential Technique

- This method is illustrated in Figure 3.3. For instance, the second message from p_3 to p_2 (which contains a timestamp $\{(3, 2)\}$) informs p_2 that the third component of the vector clock has been modified and the new value is 2.
- This is because the process p_3 (indicated by the third component of the vector) has advanced its clock value from 1 to 2 since the last message sent to p_2 .
- This technique substantially reduces the cost of maintaining vector clocks in large systems, especially if the process interactions exhibit temporal or spatial localities.

Singhal-Kshemkalyani's Differential Technique

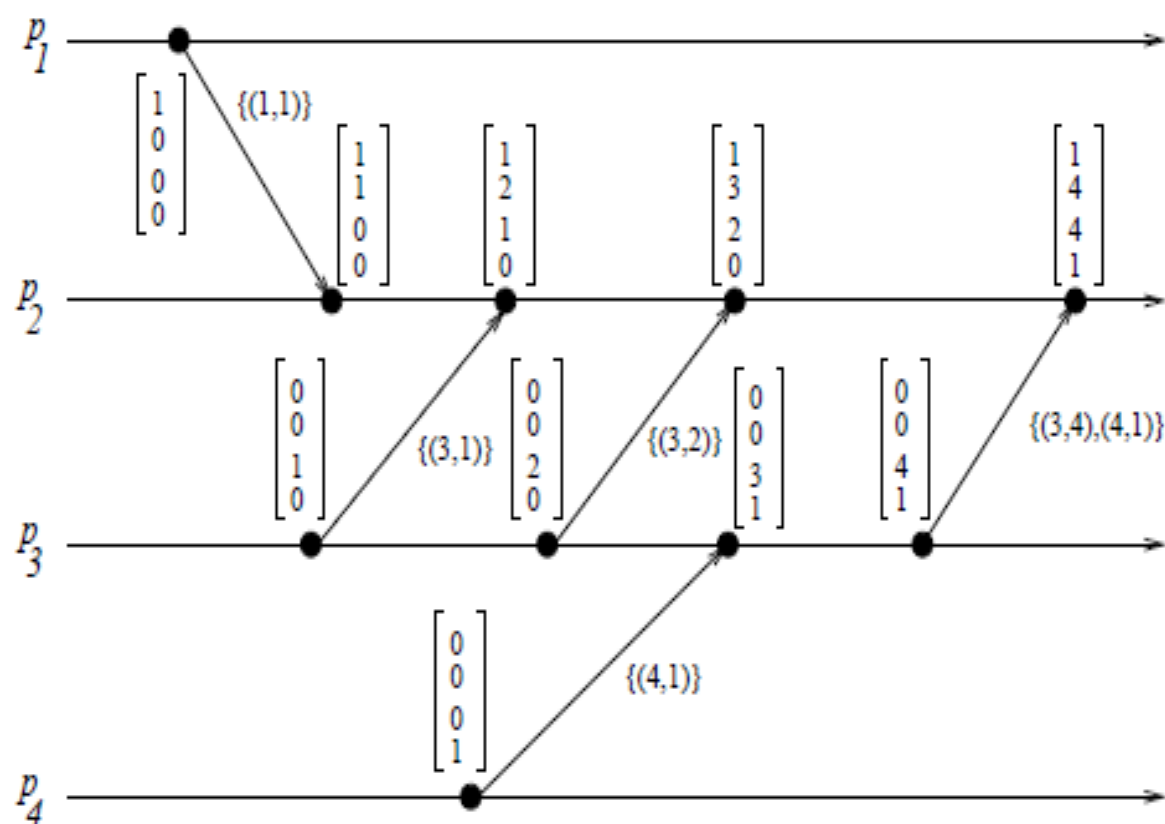


Figure 3.3: Vector clocks progress in Singhal-Kshemkalyani technique.

Matrix Time

In a system of matrix clocks, the time is represented by a set of $n \times n$ matrices of non-negative integers.

A process p_i maintains a matrix $mt_i[1..n, 1..n]$ where,

- $mt_i[i, i]$ denotes the local logical clock of p_i and tracks the progress of the computation at process p_i .
- $mt_i[i, j]$ denotes the latest knowledge that process p_i has about the local logical clock, $mt_j[j, j]$, of process p_j .
- $mt_i[j, k]$ represents the knowledge that process p_i has about the latest knowledge that p_j has about the local logical clock, $mt_k[k, k]$, of p_k .
- The entire matrix mt_i denotes p_i 's local view of the global logical time.

Matrix Time

Process p_i uses the following rules $R1$ and $R2$ to update its clock:

- $R1$: Before executing an event, process p_i updates its local logical time as follows:

$$mt_i[i, i] := mt_i[i, i] + d \quad (d > 0)$$

- $R2$: Each message m is piggybacked with matrix time mt . When p_i receives such a message (m, mt) from a process p_j , p_i executes the following sequence of actions:
 - ▶ Update its global logical time as follows:

$$(a) \quad 1 \leq k \leq n : mt_i[i, k] := \max(mt_i[i, k], mt[j, k])$$

(That is, update its row $mt_i[i, *]$ with the p_j 's row in the received timestamp, mt .)

$$(b) \quad 1 \leq k, l \leq n : mt_i[k, l] := \max(mt_i[k, l], mt[k, l])$$

- ▶ Execute $R1$.
- ▶ Deliver message m .

Matrix Time

- Figure 3.4 gives an example to illustrate how matrix clocks progress in a distributed computation. We assume $d=1$.
- Let us consider the following events: e which is the x_i -th event at process p_i , e_k^1 and e_k^2 which are the x_k^1 -th and x_k^2 -th event at process p_k , and e_j^1 and e_j^2 which are the x_j^1 -th and x_j^2 -th events at p_j .
- Let mt_e denote the matrix timestamp associated with event e . Due to message m_4 , e_k^2 is the last event of p_k that causally precedes e , therefore, we have $mt_e[i, k] = mt_e[k, k] = x_k^2$.
- Likewise, $mt_e[i, j] = mt_e[j, j] = x_j^2$. The last event of p_k known by p_j , to the knowledge of p_i when it executed event e , is e_k^1 ; therefore, $mt_e[j, k] = x_k^1$. Likewise, we have $mt_e[k, j] = x_j^1$.

Matrix Time

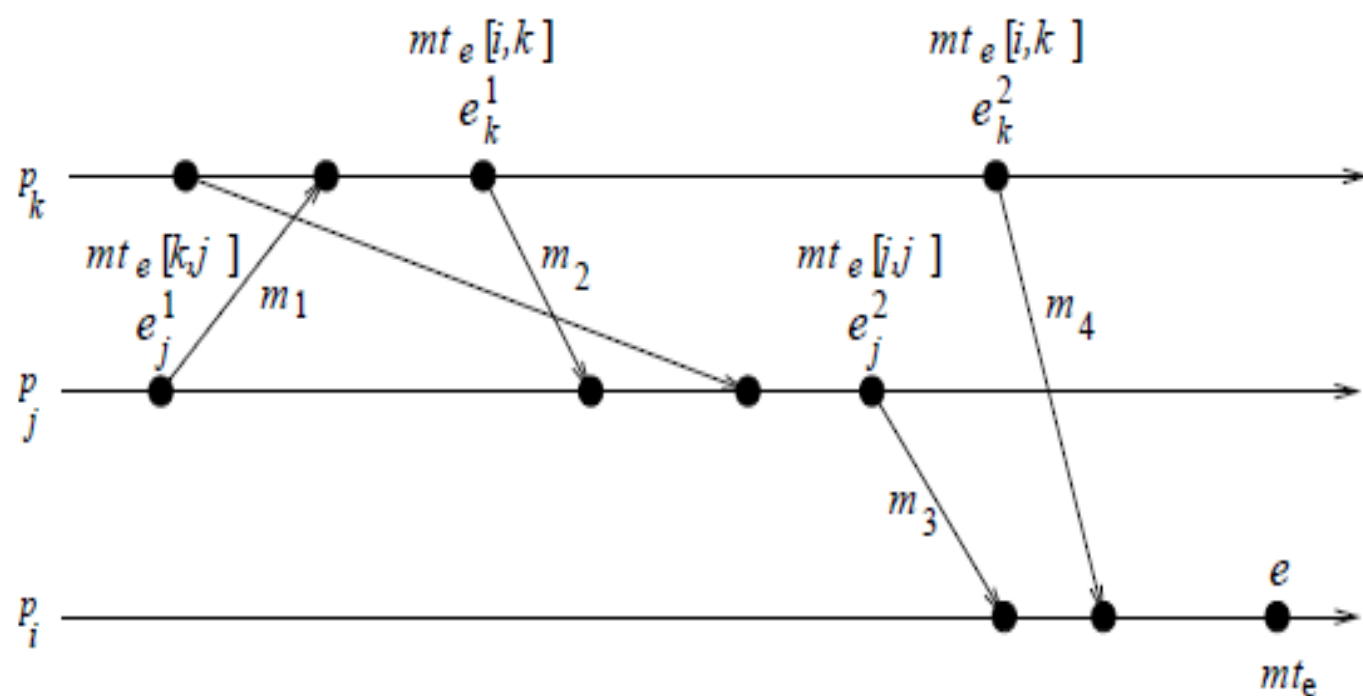


Figure 3.4: Evolution of matrix time.

Matrix Time

Basic Properties

- Vector $mt_i[i, .]$ contains all the properties of vector clocks.
- In addition, matrix clocks have the following property:
 $\min_k (mt_i[k, l]) \geq t \Rightarrow$ process p_i knows that every other process p_k knows that p_l 's local time has progressed till t .
 - ▶ If this is true, it is clear that process p_i knows that all other processes know that p_l will never send information with a local time $\leq t$.
 - ▶ In many applications, this implies that processes will no longer require from p_l certain information and can use this fact to discard obsolete information.
- If d is always 1 in the rule $R1$, then $mt_i[k, l]$ denotes the number of events occurred at p_l and known by p_k as far as p_i 's knowledge is concerned.